CS4433 – Software Engineering
Final Project Report

Moderated Learning Management System (LMS)

Student	Katleho Senqhe
Student number	202101690
Email	katlehochristophersenqhe@gmail.com
Development model	Waterfall with V-Model verification and validation
Tools	C++17, CMake, SQLite, Catch2, Git, cppcheck
GitHub URL	https://github.com/katlehochristophersenqhe-maker/CS4433-LMS.git

Abstract

This report presents the complete software engineering life cycle for a Moderated Learning Management System developed for a secondary school. The system supports four roles—Administrator, Lecturer, Head of Department (HoD) and Student—and enforces a moderated assessment workflow in which marks remain hidden until the HoD releases them. Late submissions are accepted but incur a penalty of 10 % of the maximum mark for each whole day late, up to three days. Students may resubmit before the effective deadline; only the latest version is retained. Every mark change is recorded in an immutable audit log.

The project followed the Waterfall model with V-Model verification and validation. Phase 1 produced a feasibility study, work-breakdown structure, Gantt chart and risk register. Phase 2 resolved stakeholder conflicts, produced a Software Requirements Specification containing 15 functional and 8 non-functional requirements, and documented three fully-dressed use cases. Phase 3 delivered a layered architecture, a detailed class diagram applying the Strategy, Repository and Factory patterns, behavioural models (state machine, activity and sequence diagrams) and a Third-Normal-Form SQLite schema. Phase 4 implemented the system in C++17 with CMake, SQLite and a console presentation layer. Phase 5 defined a test strategy mapped to the V-Model, automated unit and integration tests with Catch2, and system/acceptance tests derived from the use cases. Phase 6 established Git branching, a continuous-integration pipeline and static analysis. Phase 7 packaged the release with installation and user manuals. Phase 8 performed perfective maintenance for class-wide deadline extensions (release v1.1) and addressed professional, legal and ethical issues under the Lesotho Data Protection Act 2011.

The resulting system meets all Must requirements, demonstrates consistent traceability from requirements through design, code and tests, and provides a maintainable offline application suitable for the constrained school environment.

Table of Contents

1. Title page, abstract and table of contents
2. Introduction
3. Project management plan
4. Requirements engineering
5. Software design document
6. Implementation
7. Verification and validation
8. Configuration management and quality assurance
9. Deployment
10. Maintenance (change request CR1)
11. Professional, legal and ethical issues
12. Conclusion
13. References
14. Appendices

2. Introduction

Secondary schools need reliable tools to manage courses, assignments and assessment while preserving academic integrity. The Moderated Learning Management System (LMS) described in this report was developed to meet that need under realistic constraints: the software must run offline on modest hardware, support four distinct user roles, apply late-penalty rules, require Head-of-Department moderation before marks become visible to students, and keep a complete audit trail of every mark change.

The project was undertaken as the individual final assignment for CS4433 Software Engineering at the National University of Lesotho. It follows the Waterfall life-cycle model with V-Model verification and validation, as required by the brief. Each phase ends with a documented deliverable; implementation was not begun until the design had passed Gate 2.

The system is a C++17 console application that stores all data in a local SQLite database. It deliberately excludes web or mobile interfaces, automated grading, plagiarism detection and online payment so that the engineering effort can be focused on correct role-based workflow, secure persistence, design patterns and systematic testing.

This report consolidates every phase of the work into a single coherent document. It shows how stakeholder conflicts were resolved, how the architecture and design patterns support the business rules, how the implementation realises the design, how the system was verified, and how a post-release change request was analysed, implemented and regression-tested. Professional, legal and ethical considerations, particularly data protection of learner information under Lesotho law, are examined in the final technical section.

The remainder of the report is organised exactly as required by the assignment brief. Section 3 presents the project management plan, including the feasibility study, work-breakdown structure and risk register. Section

4 records the requirements-engineering work: stakeholder analysis, conflict resolution, the Software Requirements Specification and the use-case model. Section 5 contains the software design document—architecture, class diagram with design patterns, behavioural models and the normalised database. Section 6 describes the C++17 implementation, its fidelity to the design, and the security and persistence measures. Section 7 covers verification and validation according to the V-Model. Section 8 addresses configuration management and quality assurance. Section 9 deals with deployment. Section 10

analyses and implements the post-release change request CR1. Section 11 discusses professional, legal and ethical issues. Section 12 concludes, Section 13 lists the references, and Section 14 contains the appendices (traceability matrix, test-result summary, user-manual overview and the updated risk register).

Throughout the project the same system is described. Requirements, design elements, source files and test cases are kept consistent with one another. Where a design decision differs from an earlier assumption, the change is recorded and justified. The Git repository that accompanies this report contains the complete source code, the automated test suite, the continuous-integration configuration, the installation and user manuals, and the intermediate phase documents.

3. Project Management Plan

3.1 Feasibility study

A structured feasibility assessment was performed at the start of the project. Technical feasibility is high: the mandated tool chain (C++17, CMake, SQLite, Catch2 or GoogleTest, a static analyser and Git) is mature and free. The main technical risk is the correct implementation of salted password hashing and of the multi-state submission life cycle within a single developer's capacity. This risk is mitigated by producing a minimal working skeleton early and by continuous integration.

Operational feasibility is supported by a deliberately simple menu-driven console interface. Teachers are not assumed to be IT experts; numbered menus and short, clear prompts keep the cognitive load low. Economic feasibility is excellent: every required tool is open-source, so there are no licensing costs. Legal feasibility requires compliance with the Lesotho Data Protection Act 2011; the design therefore minimises personal data collection, stores only salted password hashes, and maintains an audit log of assessment decisions. Schedule feasibility is acceptable within the seven-week window provided the Waterfall gates are respected and buffer time is reserved before each gate.

Overall recommendation: the project is feasible and should proceed.

The feasibility study also considered the constraints imposed by the school environment. The IT officer noted that computers are old and the internet is unreliable. Consequently the architecture was required to be fully offline; SQLite was chosen because it needs no separate server process and stores the entire database in a single file that can be backed up by ordinary file-system tools. Performance targets (login $\leq$ 5 s with 800 student records, assignment list $\leq$ 1.5 s) were written into the non-functional requirements so that they could be verified later. These early decisions reduced the risk that the finished system would be unusable on the target hardware.

3.2 Work breakdown and schedule

The work was decomposed into nine major packages that map directly onto the phases defined in the assignment brief. Implementation (Phase 4) is the largest package because it carries 21 of the 100 marks and because a four-role, five-state system has substantial surface area. Configuration management and quality assurance are treated as continuous activities rather than a single discrete block.

WBS	Work package	Rationale / notes
1	Planning and management	Feasibility, schedule, initial risk register

2	Requirements engineering	Stakeholder analysis, conflict resolution, SRS, use cases
3	System design	Architecture, class diagram, patterns, behavioural models, database
4	Implementation	Largest package; C++17 + SQLite + design patterns
5	Verification and validation	Test plan, automated tests, system/acceptance tests, defect log
6	Configuration management & QA	Git discipline, CI, static analysis (continuous)
7	Deployment	Deployment diagram, installation guide, user manuals, v1.0 tag
8	Maintenance	CR1 impact analysis, v1.1 implementation, regression testing
9	Report, ethics, viva preparation	Runs throughout; final consolidation

A Gantt chart was prepared with dependencies, milestones (Gate 1 Requirements, Gate 2 Design, Gate 3 Testing) and a critical path that runs through Requirements → Design → Implementation → Verification. One day of buffer was placed before each gate to absorb minor overruns.

3.3 Risk management

A risk register of seven risks covering technical, project and people categories was maintained and reviewed at each phase gate. The highest-priority risks concerned correct password hashing, SQLite performance with 800 students, requirement ambiguity, and insufficient test coverage. Mitigations included early prototyping of the security layer, normalised schema design with indexes, documented conflict-resolution tables, and continuous coverage measurement.

ID	Risk	Category	Mitigation
R01	Difficulty implementing secure salted hashing	Technical	Use mature library / well-tested portable hash; prototype early
R02	SQLite performance with 800 students	Technical	3NF schema, indexes on FKs, early volume tests
R03	Ambiguous stakeholder requirements cause rework	Project	Conflict-resolution table; Gate 1 sign-off
R04	Inadequate test coverage (<70 %)	Project/Tech	Tests written alongside code; CI; coverage tools
R05	Illness, power cut or outage	People	Buffer day; offline-first Git; charged laptop
R06	Viva date compresses final preparation	People	Confirm date early; rehearse earlier if needed
R07	Late completion of Design (Gate 2)	Project	Strict Gantt adherence; buffer in Weeks 2–3

4. Requirements Engineering

4.1 Stakeholder analysis and conflict resolution

Six stakeholder groups were identified: Principal, Lecturer, Head of Department, Student, System Administrator and IT Officer. Their interests and key concerns were tabulated. The Principal prioritises simplicity and HoD control of marks; the Lecturer prioritises rapid release of marks and the ability to grant extensions; the HoD

insists on a late-penalty rule and the right to moderate every mark; the Student wants resubmission rights, written feedback and acceptance of late work; the Administrator needs full account and enrolment management; the IT Officer requires offline operation, password protection and acceptable performance with 800 students.

Three material conflicts were resolved as follows. First, the visibility of marks: the Principal's requirement that students see nothing until HoD release was adopted; lecturers may mark and write feedback, but the data remain hidden until release. Second, late-work handling: the HoD's 10 %-per-day rule (maximum three days, then zero) was adopted, while still accepting the submission so that late work is not simply rejected. Third, resubmission: students may resubmit any number of times before the effective deadline; only the latest version is retained for marking. Individual extensions were classified as Must; class-wide extensions were later added as a perfective change (Phase 8).

Additional ambiguities were clarified during the same analysis. The student representative asked for written feedback rather than a numeric mark alone; this was accepted as a Must requirement and appears in FR-09 and UC-02. The lecturer's desire for rapid mark release was subordinated to the Principal's integrity requirement, but the design still allows the lecturer to complete marking and feedback immediately; only visibility is deferred. The IT officer's performance and offline constraints were translated into measurable non-functional requirements (NFR-01, NFR-02, NFR-07) so that they could be verified rather than left as vague aspirations. The resulting set of decisions is internally consistent and is reflected uniformly in the SRS, the use-case specifications, the design models and the implemented code.

4.2 Software Requirements Specification

The SRS follows the spirit of ISO/IEC/IEEE 29148. Fifteen functional requirements and eight non-functional requirements were written, each uniquely identified, prioritised with MoSCoW and given a verification method. All Must requirements are measurable. Selected examples appear below; the complete set is retained in the project repository and is summarised in the traceability matrix (Appendix A).

ID	Requirement (summary)	Priority
FR-01	Authenticate users and grant role-based access	Must
FR-02	Administrator manages user accounts	Must
FR-03	Administrator manages departments, courses, enrolments	Must
FR-04	Lecturer creates assignment (title, deadline, max mark)	Must
FR-05	Lecturer grants individual extension	Must
FR-07	Student submits / resubmits before effective deadline	Must
FR-08	Automatic late penalty (10 %/day, max 3 days → 0)	Must
FR-09	Lecturer marks with numeric mark + written feedback	Must
FR-10	HoD moderates and releases marks	Must
FR-11	Student sees only released marks and feedback	Must
FR-13	Immutable audit log of every mark change	Must

Key non-functional requirements include: login completes in ≤ 5 s with 800 student records (NFR-01); fully offline operation (NFR-02); passwords stored only as salted hashes (NFR-03); mark changes and audit entries occur in the same transaction (NFR-04); menu-driven UI usable by non-IT staff (NFR-05); graceful recovery

from invalid input and database errors (NFR-06); C++17, CMake and a documented coding standard (NFR-08).

4.3 Use-case model

A use-case diagram covers the four primary actors (Student, Lecturer, HoD, Administrator). Include, extend and generalisation relationships are used where appropriate. Three use cases were written in fully-dressed form (preconditions, main success scenario, alternative and exception flows, postconditions and business rules).

UC-01 Submit Assignment covers on-time submission, the late path (automatic penalty calculation) and the resubmission path (replacement of the previous version). Business rule BR-01 encodes the 10 %-per-day penalty; BR-02 states that only the latest submission is retained. UC-02 Mark Submission requires a numeric mark and written feedback and leaves the mark invisible to the student. UC-03 Moderate and Release Marks requires a reason whenever a mark is changed and records the change in the audit log before the status becomes Released.

5. Software Design Document

5.1 Architecture

A layered architecture with four layers was selected: Presentation (console menus), Application / Business Logic (services that enforce permissions and business rules), Domain (core entities and pure rules) and Data Access (repository interfaces and SQLite implementations). Dependencies point strictly downward. This style gives a clear home for business rules, supports transactional audit logging, and keeps the presentation layer free of SQL and domain logic.

Model–View–Controller was considered and rejected. MVC is advantageous for rich graphical interfaces; the LMS only needs a simple console UI. A layered design provides a clearer separation of business rules and the persistence layer without the extra ceremony of a full MVC framework.

The principal components are ConsoleUI, AuthService, AdminService, LecturerService, HoDService, StudentService, the repository interfaces and their SQLite implementations, LatePenaltyStrategy and UserFactory. All database access occurs through the data-access layer using parameterised queries.

5.2 Class diagram and design patterns

An abstract User class holds common attributes (identifier, username, password hash, full name, active flag). Four concrete subclasses—Student, Lecturer, HeadOfDepartment and Administrator—provide role-specific behaviour. Polymorphism is used throughout the service layer.

Three design patterns are applied. The Strategy pattern isolates the late-penalty calculation behind the interface LatePenaltyStrategy; StandardLatePenaltyStrategy implements the school rule (10 % per whole day, maximum three days, thereafter zero). A new penalty policy can be introduced without modifying the submission service (Open/Closed Principle). The Repository / DAO pattern provides an interface for each aggregate (UserRepository, SubmissionRepository, etc.); concrete SQLite classes implement the interfaces. Services depend only on the abstractions (Dependency Inversion). The Factory pattern centralises creation of the correct User subclass, removing switch statements from the administration service.

The design observes the SOLID principles: each class has a single primary responsibility; the penalty strategy is open for extension; User subclasses are substitutable; repository interfaces are narrow; and high-level modules depend on abstractions.

Relationships and multiplicities are stated explicitly. A Department contains many Courses; a Course has many Assignments and many Enrolments; a Student has many Enrolments and many Submissions; an Assignment has many Submissions and zero or more Extensions; a Submission has zero or more AuditLogEntry records. A Lecturer is linked to the Courses they teach; a HoD is linked to one Department. These multiplicities appear

both on the class diagram and as foreign-key constraints in the database schema, ensuring that the implementation cannot violate the design.

Visibility of attributes and operations follows normal C++ practice: data members are private or protected; operations that form the public contract of a class are public; helper methods that exist only for internal implementation are private. The abstract User class declares pure virtual or virtual methods that the concrete role classes override, enabling polymorphic behaviour when a service holds a base-class pointer.

5.3 Behavioural models

A state-machine diagram defines the Submission life cycle: Submitted → Late → Marked → Moderated → Released. Resubmission is permitted only while the status is Submitted or Late and before the effective deadline. Marks remain invisible to the student until the Released state is reached.

An activity diagram with swimlanes (Student, System, Lecturer, HoD) shows the end-to-end process from submission to released mark, including the optional moderation loop and the requirement that every mark change carries a reason and an audit entry.

Sequence diagrams for UC-01 (Submit Assignment) and UC-03 (Moderate and Release Marks) illustrate the interactions among ConsoleUI, the appropriate service, the repositories, the late-penalty strategy and the database. Both the happy path and the late / resubmission / error paths are shown.

5.4 Database design

The logical model is normalised to Third Normal Form. The principal tables are User (single-table inheritance via a role column), Department, Course, Enrolment, Assignment, Extension (nullable student_id permits class-wide extensions), Submission (UNIQUE constraint on assignment_id + student_id) and Audit_Log (append-only, reason NOT NULL). Foreign keys and CHECK constraints enforce integrity. A data dictionary and the SQL schema accompany the design; the mapping from tables to domain classes is explicit, with justified differences such as the storage of final_mark on Submission for query performance while the full history resides in Audit_Log.

Single-table inheritance was chosen for the User hierarchy because authentication queries need only a single table lookup and because the number of role-specific attributes is small. The role column is constrained by a CHECK clause so that only the four permitted values can be stored. Password hash and salt columns never leave the data-access layer after a successful login; higher layers receive only a domain User object that contains no secret material. The UNIQUE constraint on (assignment_id, student_id) guarantees that resubmission replaces the previous row rather than creating duplicates, which matches business rule BR-02. Indexes are placed on foreign keys and on columns that appear in frequent WHERE clauses (status, department_id) so that the performance targets remain attainable with 800 students.

6. Implementation

6.1 Structure and fidelity to design

The source tree mirrors the layered architecture: include/lms/{domain,app,infra,ui} and corresponding src directories. Domain classes (User hierarchy, Assignment, Submission, LatePenaltyStrategy) contain no SQL. Application services (AuthService, AdminService, LecturerService, HoDService, StudentService) orchestrate use cases and enforce permissions. Infrastructure classes implement the repository interfaces against SQLite with parameterised statements. ConsoleUI is a thin presentation layer that only displays menus and forwards requests.

CMake builds the application and the Catch2 test executable. The amalgamation of SQLite is compiled as C and linked into the binary so that the system remains self-contained and offline. A documented coding standard (naming, formatting, exception usage) is followed throughout.

6.2 Key design decisions realised in code

The abstract User class and the UserFactory are implemented exactly as designed; polymorphism is used when services receive a base pointer. The Strategy pattern is realised by an abstract LatePenaltyStrategy and a concrete StandardLatePenaltyStrategy that encodes BR-01. All repositories expose interfaces; the SQLite implementations never leak into the service layer. Transactions wrap mark changes and the corresponding audit-log inserts so that a failure rolls both back (NFR-04).

6.3 Security and persistence

Plain-text passwords are never stored. A PasswordHasher component produces a salted hash (portable SHA-256 with a per-user salt; the interface is ready for a stronger algorithm such as Argon2). Verification compares the recomputed hash with the stored value. The salt and hash remain inside the data-access layer after authentication; they are not returned to higher layers.

All SQL uses parameter binding, eliminating classical injection vectors. Input validation and C++ exceptions prevent invalid data or database errors from terminating the process; user-friendly messages are shown instead (NFR-06).

6.4 Business rules implemented

Late penalty, individual and (after Phase 8) class-wide extensions, resubmission of the latest version only, marking with mandatory written feedback, HoD moderation that requires a reason, release that makes marks visible, and the immutable audit log are all present and exercised by the automated tests. Role-based access control is enforced at the service boundary; a Student cannot invoke Lecturer or HoD operations.

The console user interface is deliberately minimal. After a successful login the user is presented with a numbered menu that contains only the operations permitted by their role. Invalid menu choices and malformed numeric input are rejected with a short explanatory message; the program does not crash. The same validation philosophy is applied inside the service layer: empty submission content, marks outside the permitted range, and missing moderation reasons are all detected before any database write occurs.

Build reproducibility is achieved with CMake. A single CMakeLists.txt defines the application library targets (domain, application, infrastructure), the executable, and the Catch2 test executable. The SQLite amalgamation is compiled as C and linked statically, so the resulting binary has no external shared-library dependency on a system SQLite installation. This satisfies the offline requirement and simplifies deployment on school machines that may not have development packages installed.

Demonstration data are seeded on the first run of a new database file. Six accounts covering all four roles are created, together with sample departments, courses, assignments and submissions in various states (Submitted, Late, Marked, Released). This allows a supervisor or examiner to exercise every major flow without first having to populate the database manually.

7. Verification and Validation

7.1 Test strategy

The test strategy follows the V-Model. Unit tests verify domain classes and pure functions (late-penalty strategy, password hasher, user factory). Integration tests verify collaboration between services and the SQLite repositories (authentication, user creation). System tests exercise complete use-case flows through the console interface or service façade. Acceptance tests confirm that the Must requirements are satisfied from the stakeholder viewpoint. Entry criteria for each level include a successful build and, for higher levels, the lower-level tests already green. Exit criteria include zero open critical defects and the coverage target for the business-logic layer.

Equivalence partitioning and boundary-value analysis are applied systematically to deadlines, days-late values (0, 1, 3, 4), mark ranges (0, maxMark) and password validity. Security tests confirm that the password_hash column never contains plain text and that cross-role actions are rejected.

7.2 Automated unit and integration tests

Catch2 (single-header) is used. The test suite comprises unit tests for the late-penalty strategy, the password hasher and the user factory, together with integration tests for authentication and secure storage. After the Phase 8 change the suite also contains dedicated CR1 tests. The current run reports 65 assertions across 25 test cases, all passing. Line coverage of the business-logic layer (domain + application services) meets the 70 % target when measured with the coverage tooling described in the test plan.

7.3 System and acceptance testing

System test cases were derived directly from the fully-dressed use cases. ST-01 covers on-time submission, the late path and resubmission. ST-02 covers marking with feedback and the continued invisibility of the mark to the student. ST-03 covers moderation (with and without a mark change) and release. Exception flows (empty content, out-of-range mark, missing reason) are exercised and produce user-friendly messages without crashing. A defect log records severity, root cause, fix and retest result; no critical defects remain open at the time of writing.

Boundary-value analysis was applied rigorously to the late-penalty rule. Test cases exercise zero days late (no penalty), one day late (10 % deduction), three days late (30 % deduction) and four or more days late (mark set to zero). Equivalent partitions for password handling include correct password, incorrect password, unknown username and empty password field. Role-based access tests attempt cross-role operations (for example a Student calling a Lecturer service method) and confirm that the operation is rejected before any side-effect occurs.

Coverage measurement is performed with the compiler's instrumentation flags and the lcov tool on platforms that support them. The reported figure for the business-logic layer (domain classes plus application services) meets or exceeds the 70 % line-coverage target required by the brief. The presentation layer and the generated SQLite amalgamation are excluded from the coverage target because they contain little decision logic of their own.

8. Configuration Management and Quality Assurance

8.1 Configuration management

The project is under Git with a documented branching strategy (feature branches merged via pull request onto main). Commits carry meaningful messages. Releases are tagged (v1.0 for the initial delivery; v1.1 prepared for the CR1 change). A GitHub Actions workflow builds the project and runs the Catch2 suite on every push, providing continuous integration evidence.

8.2 Quality assurance

cppcheck (and optionally clang-tidy) is run against the source tree; findings are addressed or explicitly justified. Code metrics (lines of code, cyclomatic complexity of key functions) are recorded. A self code-review checklist (naming, exception safety, parameterised SQL, absence of magic numbers, consistency with the design) is applied; changes made as a result of the review are noted in the repository history.

9. Deployment

A UML deployment diagram shows a single school workstation (or a small local network share) hosting the executable and the SQLite database file. No application server or external network service is required.

The installation guide is written for a school technician: install a C++17 toolchain if building from source, or unpack a pre-built release archive; place the executable and the schema file in a writable directory; run the program once to create the database; optionally load the seeded demonstration accounts. The system is fully offline thereafter.

Role-specific user manuals (Administrator, Lecturer, HoD, Student) list every menu option, the expected inputs and the business rules that the user will encounter. Release notes for v1.0 summarise the implemented Must requirements and known limitations (console-only UI, single-file deployment).

10. Maintenance (Change Request CR1)

10.1 Classification and impact analysis

After release v1.0 the school submitted CR1: “A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student’s assignment list.”

CR1 is classified as perfective maintenance. It improves the product to meet an additional user need; it does not correct a defect (corrective) nor adapt the system to a changed environment (adaptive). The v1.0 schema already permitted a NULL student_id on the Extension table, so the data model required only minor clarification.

Impact analysis identified the affected artefacts: the extension-related functional requirements and business rules; the Lecturer and Student use-case descriptions; the class diagram (new grantClassExtension operation); the sequence and activity diagrams that show extension handling; LecturerService, StudentService and ConsoleUI; the Extension repository (already NULL-capable); the student assignment-list view; and the test suite. No schema migration is required for a fresh install; a transactional migration path is provided for legacy databases whose student_id column was defined as NOT NULL.

10.2 Implementation (release v1.1)

LecturerService::grantClassExtension validates the request and persists an Extension row with a null student identifier. The student assignment list now displays the effective deadline (the later of the original deadline and any applicable individual or class-wide extension); individual extensions take precedence. The console menu for the lecturer gains a “Grant class extension” option. All changes remain consistent with the layered architecture and the existing design patterns.

10.3 Regression testing

New tests in test_cr1_extensions.cpp verify class-wide persistence, visibility of the effective deadline to the student, correct late-penalty calculation when a class-wide extension is present, precedence of individual over class-wide extensions, and validation failures. The entire existing suite is re-run; 65 assertions across 25 test cases pass. The defect log records no regressions.

11. Professional, Legal and Ethical Issues

The system processes personal information of learners (names, identifiers, assessment records). Under the Lesotho Data Protection Act 2011 the processing must be lawful, fair and transparent; data must be adequate, relevant and not excessive; and appropriate technical measures must protect against unauthorised access. The design addresses these obligations by storing only the data required for assessment, by never storing plain-text passwords, by restricting each role to the operations it needs, and by recording every mark change in an immutable audit log. The audit trail supports accountability if a mark is later disputed.

Security of assessment records is further strengthened by role-based access control and by the requirement that a non-empty reason accompany any moderation change. Accessibility for users with disabilities is limited by the console interface; future work could add a simple graphical front-end or screen-reader-friendly output. The late-penalty rule itself raises a fairness question: a uniform 10 %-per-day deduction up to three days treats all students equally once the deadline has passed, while still allowing individual (and, after CR1, class-wide) extensions for documented need. The rule is therefore transparent and consistently applied, which supports procedural fairness.

Professionally, the project demonstrates adherence to a documented life cycle, version control, automated testing and honest defect reporting. Claims about functionality are limited to what the test suite and the acceptance scenarios actually exercise.

12. Conclusion

The Moderated Learning Management System was taken through the full software engineering life cycle required by CS4433. Stakeholder conflicts were resolved in favour of academic integrity (HoD moderation before release) while still accommodating late work, resubmission and written feedback. A layered architecture, three classical design patterns and a normalised SQLite schema produced a maintainable design that maps cleanly onto the C++17 implementation. Automated tests, continuous integration and static analysis provide evidence of quality. The perfective change CR1 (class-wide extensions) was analysed, implemented and regression-tested without disturbing existing behaviour. Professional and legal obligations, especially the protection of learner data, were considered explicitly and reflected in the design.

Remaining limitations are the console-only user interface and the single-workstation deployment model. Future enhancements could include a lightweight graphical shell, richer reporting for the HoD, and stronger password hashing (Argon2). Within the stated scope the system satisfies all Must requirements and is ready for supervised use in the target school environment.

The project also demonstrated the practical value of the V-Model. Because each test level was defined against a specific development artefact, defects were discovered close to their origin. Unit tests caught errors in the late-penalty arithmetic and in the password hasher before those components were integrated. Integration tests exposed a transactional issue in the audit-log path that was corrected while the design was still fresh in mind. System tests derived from the use-case specifications confirmed that the end-to-end flows, including the exception paths, behaved as stakeholders expected. The same discipline made the Phase 8 perfective change straightforward: the impact analysis listed every affected artefact, the implementation stayed inside the existing architectural boundaries, and the regression suite confirmed that no previously working behaviour had been broken.

From a personal learning perspective the assignment required the continuous exercise of engineering judgement: when to accept a stakeholder statement, when to mediate a conflict, which architectural style best fitted the constraints, which design patterns would keep the code open for future change, and how much testing was enough to give confidence without exhausting the available time. The resulting artefacts—plans, requirements, design models, source code, tests and this report—form a coherent chain of evidence that the Moderated LMS was built systematically rather than opportunistically.

13. References

Boehm, B.W. (1981) Software engineering economics. Englewood Cliffs, NJ: Prentice-Hall.

Boehm, B.W. (1991) ‘Software risk management: principles and practices’, IEEE Software, 8(1), pp. 32–41.

Cockburn, A. (2001) Writing effective use cases. Boston: Addison-Wesley.

Fowler, M. (2002) Patterns of enterprise application architecture. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1995) Design patterns: elements of reusable object-oriented software. Reading, MA: Addison-Wesley.

Government of Lesotho (2011) Data Protection Act, 2011 (Act No. 5 of 2011). Maseru: Government Printer.

IEEE (2011) ISO/IEC/IEEE 29148:2011 Systems and software engineering — Life cycle processes — Requirements engineering. New York: IEEE.

Martin, R.C. (2017) Clean architecture: a craftsman’s guide to software structure and design. Boston: Prentice Hall.

Pressman, R.S. and Maxim, B.R. (2015) Software engineering: a practitioner’s approach. 8th edn. New York: McGraw-Hill Education.

Project Management Institute (2021) A guide to the project management body of knowledge (PMBOK guide). 7th edn. Newtown Square, PA: Project Management Institute.

Sommerville, I. (2016) Software engineering. 10th edn. Harlow: Pearson.

Wiegers, K. and Beatty, J. (2013) Software requirements. 3rd edn. Redmond: Microsoft Press.

Catch2 documentation. Available at: https://github.com/catchorg/Catch2 (Accessed: November 2026).

14. Appendices

Appendix A – Requirements Traceability Matrix (excerpt)

The full matrix links every functional requirement to its use case, design elements (classes, diagrams, tables), source files and test cases. A representative excerpt is shown below; the complete matrix is maintained in the project repository.

FR	Use case	Design / class	Source / test
FR-01	Login	AuthService, User hierarchy	AuthService.cpp; IT-AUTH-*
FR-07	UC-01	StudentService, Submission	StudentService.cpp; ST-01
FR-08	UC-01 (late)	LatePenaltyStrategy	LatePenaltyStrategy.cpp; UT-LP-*
FR-09	UC-02	LecturerService, AuditLog	LecturerService.cpp; ST-02
FR-10	UC-03	HoDService, state machine	HoDService.cpp; ST-03
FR-13	UC-02/03	Audit_Log table, repository	SqliteAuditRepository; audit tests

Table 4. Traceability matrix (excerpt)

Appendix B – Test results summary

Catch2 run (post-CR1): 25 test cases, 65 assertions, 0 failures. Coverage of domain + application services ≥ 70 %. System tests ST-01, ST-02 and ST-03 executed successfully, including exception flows. Defect log contains no open critical or major defects.

Appendix C – User manuals (summary)

Full role-specific manuals are supplied in the repository (docs/USER_MANUALS.md). They list every menu option for Administrator, Lecturer, HoD and Student, together with the business rules the user will encounter. Demonstration accounts (admin/admin123, lec1/lec123, hod1/hod123, stu1/stu123, \ldots) are seeded on first run.

Appendix D – Updated risk register

All seven risks identified in Phase 1 were reviewed at each subsequent gate. R01 (hashing) and R04 (coverage) were reduced to low residual risk after the security prototype and the automated suite were in place. R03 (requirement ambiguity) was closed after Gate 1. The residual risks (illness, viva timing) remain monitored but have not materialised.